

Nonlinear Programming

- Nonlinear programming (NLP) is a branch of mathematical optimization that deals with problems where the objective function or constraints are nonlinear.
- **LP vs NLP:** linear programming assumes constant marginal contributions and fixed input requirements. Nonlinear programming lets those change with the decisions we make.
 - How much fertilizer should a farmer apply? Yield gains diminish as fertilizer increases. Profit depends on a curved yield response.
 - How much should a firm invest in R&D? The marginal returns to R&D are decreasing, and the cost of R&D may increase with scale.
 - How should a producer allocate land across risky crops? The expected return is nonlinear in the allocation, and the risk depends on the covariance of returns across crops.

1 Unconstrained Optimization

1.1 Motivation

- Unconstrained optimization involves finding the maximum or minimum of a function without any restrictions on the variable values.
- Many economic models can first be understood in a frictionless, unconstrained environment before constraints are introduced.

1.1.1 Example

- **Profit maximization by a firm:**
A monopolist choosing output q to maximize profits:

$$\max_q \pi(q) = p(q)q - C(q)$$

where $p(q)$ is the inverse demand function and $C(q)$ is the cost function. If there are no capacity or policy restrictions, this is an unconstrained problem.

- Plot to build intuition:
 - Draw the demand curve and show how we are calculating the area of the rectangle under the curve (revenue). Assume linear $p(q) = a - bq$.
 - Then draw the cost curve. Assume constant marginal cost $C(q) = cq$.
 - Profit is the difference

If demand is linear and cost is linear, why isn't profit linear? As we sell another unit, what happens to revenue from the units we were already selling?

1.2 Mathematical Setup

The general unconstrained optimization problem can be written as:

$$\max_{x \in \mathbb{R}^n} f(x)$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is the objective function.

- The solution x^* is called an optimum if $f(x^*) \geq f(x)$ for all x in some neighborhood of x^* .

1.2.1 Assumptions for tractability

- $f(x)$ is continuous and differentiable at least once (so gradients exist).
- For stronger results, $f(x)$ is twice differentiable (so the Hessian exists).
- The function is well-behaved: bounded above (for maximization problems) or below (for minimization problems).

In economics, many standard functional forms (Cobb-Douglas, CES, quadratic, log-linear) satisfy these assumptions, making unconstrained optimization analytically tractable.

1.3 Univariate setup

If $f : \mathbb{R} \rightarrow \mathbb{R}$, the problem reduces to finding the optimal point x^* such that:

1.3.1 First-Order Condition

$$f'(x^*) = 0$$

What is the graphical intuition behind the first-order condition? Suppose our candidate max yields a positive FOC, which direction should we move?

1.3.2 Second-Order Condition

- $f''(x^*) < 0 \implies$ local maximum – evaluated at the critical point x^*
- $f''(x^*) > 0 \implies$ local minimum
- $f''(x^*) = 0 \implies$ inconclusive (could be inflection point, higher-order test needed; e.g., $f(x) = x^3$ at $x = 0$)

1.3.3 Example: A quadratic profit function

Return to our example. Set up the max problem

$$\pi(q) = aq - bq^2 - cq, \quad a, b, c > 0$$

The first-order condition is:

$$\pi'(q) = a - 2bq - c = 0 \implies q^* = \frac{a - c}{2b}$$

We can verify this is a local maximum by checking the second-order condition.

The second-order condition is:

$$\pi''(q) = -2b < 0 \quad (\text{since } b > 0)$$

1.3.4 Try yourself

Consider the profit-maximization problem:

$$\max_q \pi(q) = 100 - (q - 3)^4.$$

Find the profit-maximizing quantity and show that it is indeed a maximum.

1.4 Multivariate Optimization

If $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the optimization problem is:

$$\max_{x \in \mathbb{R}^n} f(x_1, x_2, \dots, x_n)$$

1.4.1 Gradient and First-Order Conditions

$$\nabla f(x^*) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(x^*) \\ \frac{\partial f}{\partial x_2}(x^*) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x^*) \end{bmatrix} = 0$$

$\nabla f(x^*)$ is the **gradient vector** of first derivatives. It is also called the **Jacobian** in the multivariate case.

Suppose the gradient vector is $[0, 5]$. Have we found an optimum because one marginal return is zero? What small change would improve the objective?

1.4.2 Hessian and Second-Order Conditions

Let $H(x)$ denote the Hessian matrix of second derivatives:

$$H(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \dots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

If the objective curves downward in each variable separately, is that enough to establish a maximum? Could changing both variables together improve it? Consider $f(x, y) = -x^2 - y^2 + 4xy$.

One way to classify the Hessian is by examining the **leading principal minors** — the determinants of the top-left $k \times k$ submatrices ($\det(H) = f_{xx}f_{yy} - (f_{xy})^2$, more than two vars gets complicated). Let H_k denote the $k \times k$ leading principal minor.

- If $H(x^*)$ is **negative definite** $\implies$ local maximum. The sequence of leading principal minors alternates in sign, starting with negative. $\det(H_1) < 0$, $\det(H_2) > 0$, etc.
- If $H(x^*)$ is **negative semi-definite** $\implies$ local maximum or saddle point. Determinant is non-negative. All odd-order leading principal minors are non-positive, and all even-order leading principal minors are non-negative. $\det(H_1) \leq 0$, $\det(H_2) \geq 0$, etc.
- If $H(x^*)$ is **positive definite** $\implies$ local minimum. All leading principal minors are positive. $\det(H_1) > 0$, $\det(H_2) > 0$, etc.
- If $H(x^*)$ is **positive semi-definite** $\implies$ local minimum or saddle point. All leading principal minors are non-negative. $\det(H_1) \geq 0$, $\det(H_2) \geq 0$, etc.
- If $H(x^*)$ is **indefinite** $\implies$ saddle point. The leading principal minors do not follow a consistent sign pattern.

1.5 Multivariate Example: Two-Variable Unconstrained Maximization

Consider the problem:

$$\max_{x, y} f(x, y) = 100x + 150y - x^2 - y^2 - xy$$

Step 1: First-Order Conditions

Compute the partial derivatives:

$$\frac{\partial f}{\partial x} = 100 - 2x - y$$

$$\frac{\partial f}{\partial y} = 150 - 2y - x$$

Set both equal to zero:

$$100 - 2x - y = 0$$

$$150 - x - 2y = 0$$

Solve the system:

From the first equation, $y = 100 - 2x$.

Substitute into the second:

$$150 - 2(100 - 2x) - x = 0 \implies 150 - 200 + 4x - x = 0 \implies 3x - 50 = 0.$$

Or Gaussian elimination, multiplying the first row by -2:

$$(4x - x) + (150 - 200) = 0 \implies 3x - 50 = 0.$$

So $x^* = \frac{50}{3} \approx 16.67$.

Then $y^* = 100 - 2(16.67) = 66.67$.

Step 2: Second-Order Conditions

Hessian matrix:

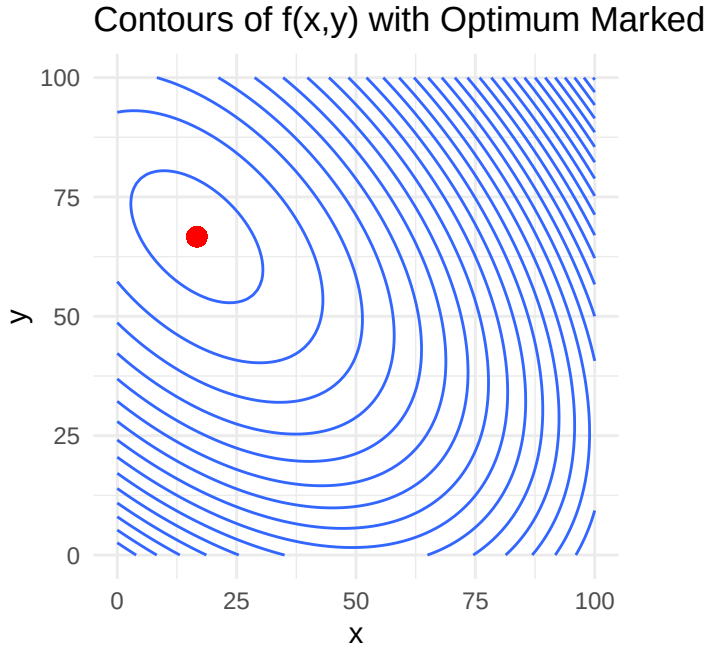
$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix} = \begin{bmatrix} -2 & -1 \\ -1 & -2 \end{bmatrix}$$

Compute its leading principal minors:

- Top-left element $|H_1| = -2 < 0$
- Determinant $|H_2| = (-2)(-2) - (-1)^2 = 4 - 1 = 3 > 0$

So H is **negative definite** $\rightarrow$ local maximum.

Step 3: Visualizing in R



1.6 Global vs. Local Optimality

Finding an optimum is not enough to guarantee that it is the **best possible solution**. We need to distinguish between **local** and **global** optima.

1.6.1 Local optimum

A point x^* is a **local maximum** if there exists a neighborhood $\mathcal{N}(x^*)$ such that:

$$f(x^*) \geq f(x) \quad \forall x \in \mathcal{N}(x^*).$$

Similarly, x^* is a **local minimum** if:

$$f(x^*) \leq f(x) \quad \forall x \in \mathcal{N}(x^*).$$

Local optima are determined by **first- and second-order conditions**:

- FOC: $\nabla f(x^*) = 0$
- SOC: Hessian conditions (positive/negative definiteness)

These tests, however, only ensure optimality relative to nearby points.

1.6.2 Global optimum

A point x^* is a **global maximum** if:

$$f(x^*) \geq f(x) \quad \forall x \in \mathbb{R}^n.$$

A point x^* is a **global minimum** if:

$$f(x^*) \leq f(x) \quad \forall x \in \mathbb{R}^n.$$

Global optimality requires stronger conditions about the **shape** of the objective function:

- If $f(x)$ is **strictly concave**, then any local maximum is also the global maximum.

- If $f(x)$ is **strictly convex**, then any local minimum is also the global minimum.

This is why assumptions of concavity and convexity are central in economics: they allow us to equate **marginal conditions** (first-order conditions) with **global results**.

1.6.3 Illustration

- A quadratic profit function

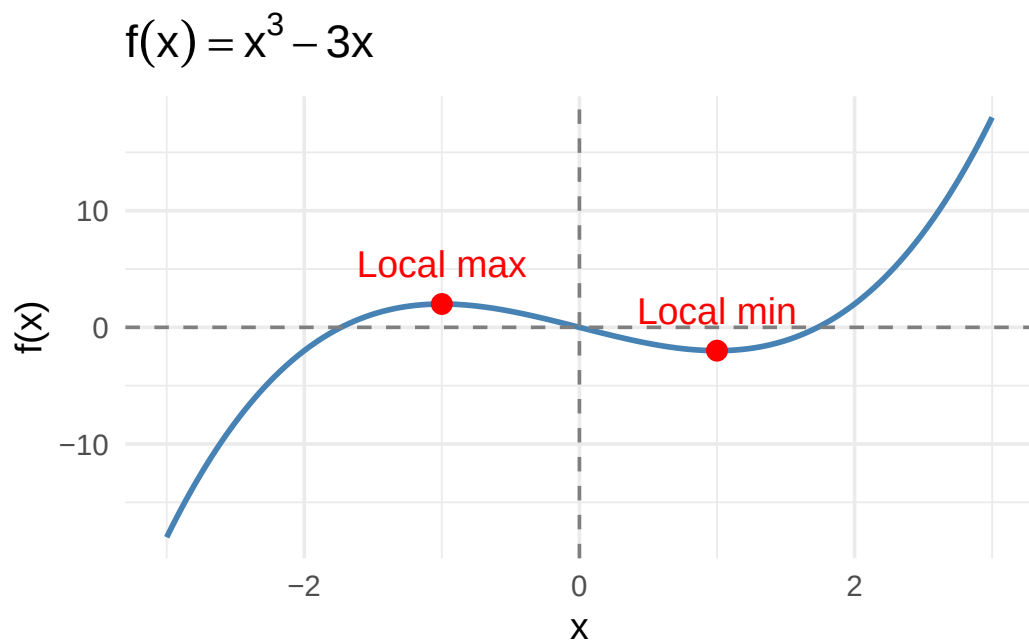
$$\pi(q) = aq - bq^2, \quad a, b > 0$$

is globally concave. The FOC $a - 2bq = 0$ and SOC $-2b < 0$, so π is globally concave.

- A non-concave function such as

$$f(x) = x^3 - 3x$$

has two local extrema ($x = -1, x = 1$). No global because as $x \rightarrow -\infty, f(x) \rightarrow -\infty$ and as $x \rightarrow \infty, f(x) \rightarrow \infty$. The FOC and SOC identify local points, but without concavity the global solution is not guaranteed.



1.6.4 Economic interpretation

- **Local optimality** corresponds to an agent being at a point where no *small deviation* is profitable.
- **Global optimality** ensures that the agent is making the *best possible choice* over the entire feasible set.

Economists typically assume concavity (utility, production, profit functions) to avoid the difficulty of distinguishing between local and global solutions. This ensures that solving the FOC is sufficient for finding the true economic optimum.

2 Numeric Foundations

- Numerical methods provide approximate solutions when analytical approaches are not feasible.
- They can work even when the function is not differentiable.
- Topics include rootfinding, gradient descent methods, and convergence criteria.

2.1 FOC to Rootfinding

Many optimization and equilibrium problems in economics reduce to **solving equations**.

$$\max_x f(x) \Rightarrow f'(x^*) = 0$$

this tells us what the solution looks like; it doesn't necessarily tell us how to find it.

For $f(x) = -(x-4)^2 + 10$, solving $f'(x) = 0$ analytically is trivial. But for more complicated functions, we may not be able to solve the FOC analytically. In these cases, we can use **numerical methods** to find a solution.

We want to find a solution x^* such that:

$$f(x^*) = 0$$

Where:

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$
- $x^* \in \mathbb{R}^n$

This is the most general formulation. Economic examples include:

- Satisfying first-order conditions in maximization problems
- Finding market-clearing prices

i Quick digression on Fixed-Point Problems

A **fixed point** of a function $g(x)$ is a solution to:

$$x = g(x)$$

This is equivalent to rootfinding if we define $f(x) = x - g(x)$. Many economic models are naturally written this way:

- Value function iteration in dynamic programming
- Nash equilibria in games

Fixed-point methods often rely on contraction mappings, which guarantee convergence under certain conditions. More on this in dynamic programming.

2.2 Bisection Method

The **bisection method** is a simple and robust way to solve nonlinear equations of the form:

$$f(x) = 0$$

It is only applicable to **univariate** problems but serves as an intro to how numerical solvers work.

2.2.1 Requirements

To apply bisection:

- The function $f(x)$ must be **continuous** on the interval $[a, b]$

- The function must change sign over the interval: $f(a) \cdot f(b) < 0 \rightarrow$ This guarantees a root exists in $[a, b]$ by the **Intermediate Value Theorem**.

How does it guarantee this? either the a or b evaluates to negative and the other positive, so by the IVT there must be a point in between where it crosses zero.

2.2.2 Algorithm

1. Check that $f(a) \cdot f(b) < 0$. If not, bisection cannot proceed.
2. Compute the midpoint: $c = \frac{a+b}{2}$
3. Evaluate $f(c)$:
 - If $f(c) = 0$, stop — you found the root
 - If $f(c)$ has the same sign as $f(a)$, replace a with c
 - Otherwise, replace b with c
4. Repeat steps 2–3 until the interval is sufficiently small.

2.2.3 Example

Solve: $f(x) = x^2 - 2 = 0$

We know $x = \sqrt{2} \approx 1.4142$ is the root.

Start with $a = 1$, $b = 2$: $f(1) = -1$, $f(2) = 2$, so the root is bracketed.

Each iteration cuts the interval in half:

Iteration	a	b	$c = \frac{a+b}{2}$	$f(c)$
1	1.0	2.0	1.5	0.25
2	1.0	1.5	1.25	-0.4375
3	1.25	1.5	1.375	-0.109375
...	...	...	...	...

After several iterations, we get very close to the root.

2.2.4 Convergence Criteria

The bisection method illustrates an important question in all numerical methods:

When do we stop iterating?

There's no exact answer in most cases — so we stop when we're “close enough.”

There are two common ways to define this:

1. Interval is small enough
 - We stop when: $|b - a| < \delta$. This ensures we're zoomed in tightly on the root. In bisection, this is the **primary** stopping rule.
2. Function value is small
 - Alternatively, we can stop when: $|f(c)| < \epsilon$. This ensures the function value at our current guess is nearly zero — a general-purpose convergence test used in many methods.

Why Both?

- In some methods, the updates $x^{(k+1)} - x^{(k)}$ may become small **even when the function value isn't** (e.g. at flat spots).

2.2.5 Multiple Roots

If the function has multiple roots in the interval, bisection will find one of them, but which one depends on the initial interval. If you need a specific root, you may need to adjust your interval accordingly. You could sample the domain to find intervals that bracket different roots.

Note also that if the function just touches zero (e.g., $f(x) = (x-1)^2$ at $x = 1$), bisection may fail since there is no sign change.

How might you improve the bisection method to find the root faster?

2.3 Newton's Method

Newton's method is a fast and widely used approach to optimizing nonlinear functions. Since optimization involves finding the roots of the first derivative, the method is applied to functions of the form:

$$f(x) = 0$$

It uses both the function **and its derivatives** to guide the search for the root. This makes it much faster than bisection or fixed-point iteration — but also more sensitive to where you start.

2.3.1 The update rule

The Newton update rule is:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Take the local linear approximation of $f(x)$:

$$L(x) = f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)})$$

We find the zero of this line by solving $L(x) = 0$, which gives:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

This is the heart of Newton's method: each step uses the slope of $f(x)$ to make a **smart jump** toward the root, rather than guessing blindly.

Intuition

Imagine standing on the curve of $f(x)$ at the point $(x^{(k)}, f(x^{(k)}))$. You draw the tangent line at that point. The next guess, $x^{(k+1)}$, is where this tangent line crosses the x-axis. This process is repeated, using the new point to draw a new tangent line, and so on, until you get sufficiently close to the root.

i Newton for optimization

To optimize an objective f , we instead solve its first-order condition, $f'(x) = 0$. Define $g(x) = f'(x)$ and apply the same Newton formula to g :

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

We optimize by finding the root of the derivative. This is the basis for **Newton-Raphson optimization**.

2.3.2 Newton's method assumptions

1. $f(x)$ is **differentiable** in the neighborhood of the root.
2. The derivative $f'(x)$ is **not zero** at the root (to avoid division by zero).
3. The initial guess $x^{(0)}$ is **sufficiently close** to the actual root for convergence.

4. $f(x)$ is sufficiently **smooth** (e.g., continuous second derivative) near the root to ensure good approximation by the tangent line.

2.3.3 Example

Let's revisit the problem $f(x) = x^2 - 2$

We have:

- $f(x) = x^2 - 2$
- $f'(x) = 2x$

Then Newton's update is:

$$x^{(k+1)} = x^{(k)} - \frac{x^{(k)^2} - 2}{2x^{(k)}}$$

Start with $x^{(0)} = 1$:

- $x^{(1)} = 1 - \frac{1^2 - 2}{2 \cdot 1} = 1 + 0.5 = 1.5$
- $x^{(2)} = 1.5 - \frac{1.5^2 - 2}{2 \cdot 1.5} \approx 1.4167$
- $x^{(3)} \approx 1.4142$

This converges very quickly to $\sqrt{2}$ — much faster than bisection.

```
library(ggplot2)

# Define the function and its derivative
f <- function(x) x^2 - 2
f_prime <- function(x) 2 * x

# Choose initial guess
x0 <- 1.0
x1 <- x0 - f(x0) / f_prime(x0) # first Newton update

# Build tangent line at x0
tangent <- function(x) f(x0) + f_prime(x0) * (x - x0)

# Create data for plotting
x_vals <- seq(0.5, 2, length.out = 300)
plot_df <- data.frame(x = x_vals, fx = f(x_vals), tangent = tangent(x_vals))

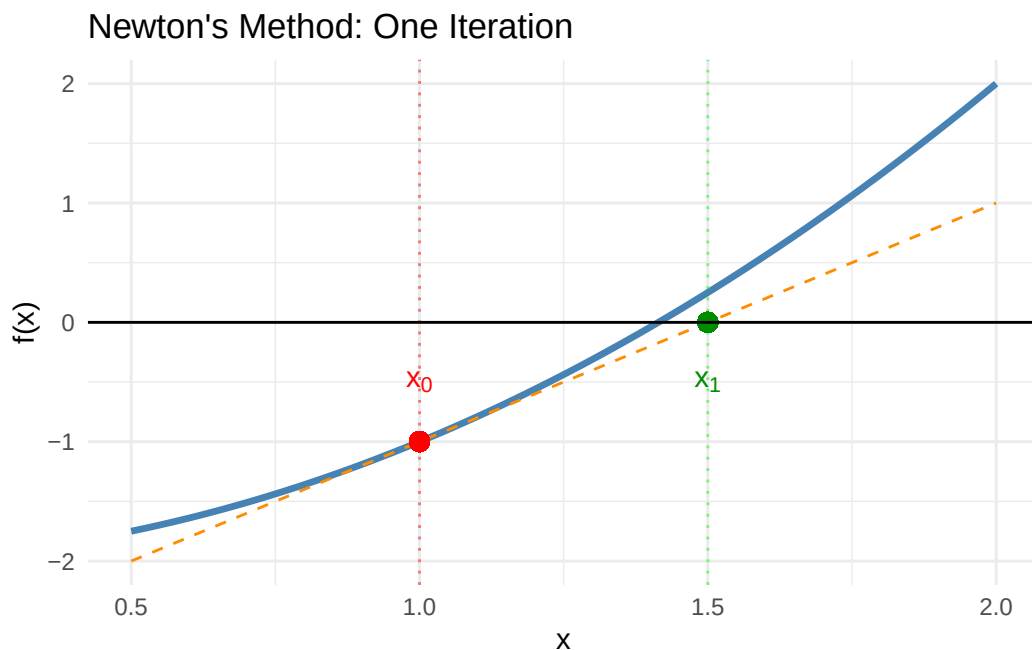
# Plot function and tangent
ggplot(plot_df, aes(x = x)) +
  geom_line(aes(y = fx), color = "steelblue", size = 1.2) +
  geom_line(aes(y = tangent), color = "darkorange", linetype = "dashed") +
  geom_vline(xintercept = x0, linetype = "dotted", color = "red", alpha=.5) +
  geom_vline(xintercept = x1, linetype = "dotted", color = "green", alpha=.5) +
  geom_point(aes(x = x0, y = f(x0)), color = "red", size = 3) +
  geom_point(aes(x = x1, y = 0), color = "green4", size = 3) +
  geom_hline(yintercept = 0, linetype = "solid") +
  annotate("text", x = x0, y = f(x0) + 0.5, label = "x[0]", parse = TRUE, color = "red") +
  annotate("text", x = x1, y = -0.5, label = "x[1]", parse = TRUE, color = "green4") +
  labs(
    title = "Newton's Method: One Iteration",
    y = "f(x)",
    x = "x"
```

```
) +  
theme_minimal()
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

Warning in geom_point(aes(x = x0, y = f(x0)), color = "red", size = 3): All aesthetics have length 1, but
i Please consider using `annotate()` or provide this layer with data containing
a single row.

Warning in geom_point(aes(x = x1, y = 0), color = "green4", size = 3): All aesthetics have length 1, but
i Please consider using `annotate()` or provide this layer with data containing
a single row.



2.3.4 Convergence Properties

Newton's method has **quadratic convergence** when it works:

- The number of correct digits roughly **doubles** each iteration

But it can also:

- **Diverge** if $f'(x)$ is small or zero
- Converge to the **wrong root** if the starting point is poorly chosen
- Be unstable near points of inflection

2.3.5 Stopping Criteria

Same logic as before, but more care needed due to speed and sensitivity:

- Change in x : $|x^{(k+1)} - x^{(k)}| < \epsilon$
- Residual is small: $|f(x^{(k)})| < \epsilon$

In practice, it's common to use both.

2.3.6 When to Use Newton's Method

- Works best when you have access to an **analytic derivative** $f'(x)$
- Very effective for well-behaved, smooth functions
- Poor choice if $f'(x)$ is expensive to compute or not available

2.4 Quasi-Newton Methods

When the derivative $f'(x)$ is not available or too costly to compute, we can use **quasi-Newton methods**. These methods approximate the derivative using finite differences or other techniques.

2.4.1 Finite Difference Approximation

A simple way to approximate the derivative is using finite differences: $f'(x) \approx \frac{f(x^k) - f(x^{k-1})}{x^k - x^{k-1}}$

You can then plug this approximation into the Newton update formula: $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$

This approach is called the **secant method** in one dimension. You can substitute the finite difference approximation for the derivative in the Newton update.

2.5 Gradient descent/ascent

What if I know the slope but don't want to calculate or use curvature?

Returning to optimization, Newton's method applied to the FOC uses $x_{k+1} = x_k - f'(x_k)/f''(x_k)$. **Gradient descent/ascent** uses only the slope of the objective, with a positive **learning rate (step size)** α_k :

$$\text{Minimization: } x_{k+1} = x_k - \alpha_k f'(x_k),$$

$$\text{Maximization: } x_{k+1} = x_k + \alpha_k f'(x_k).$$

The sign chooses downhill or uphill; α_k scales the move. The actual distance moved is $\alpha_k |f'(x_k)|$.

Use the same objective as before: maximize $f(x) = -(x-4)^2 + 10$, whose slope is $f'(x) = -2(x-4)$ and whose maximum is at $x^* = 4$. Starting from $x_0 = 0$, a constant step size gives $x_{k+1} = x_k - 2\alpha(x_k - 4)$:

- **Too small** ($\alpha = 0.05$): progress is slow.
- **Reasonable** ($\alpha = 0.4$): iterates converge quickly to 4.
- **Too large** ($\alpha = 1$): iterates overshoot and oscillate between 0 and 8.
- **Much too large** ($\alpha = 1.2$): oscillations grow and the iterates diverge.

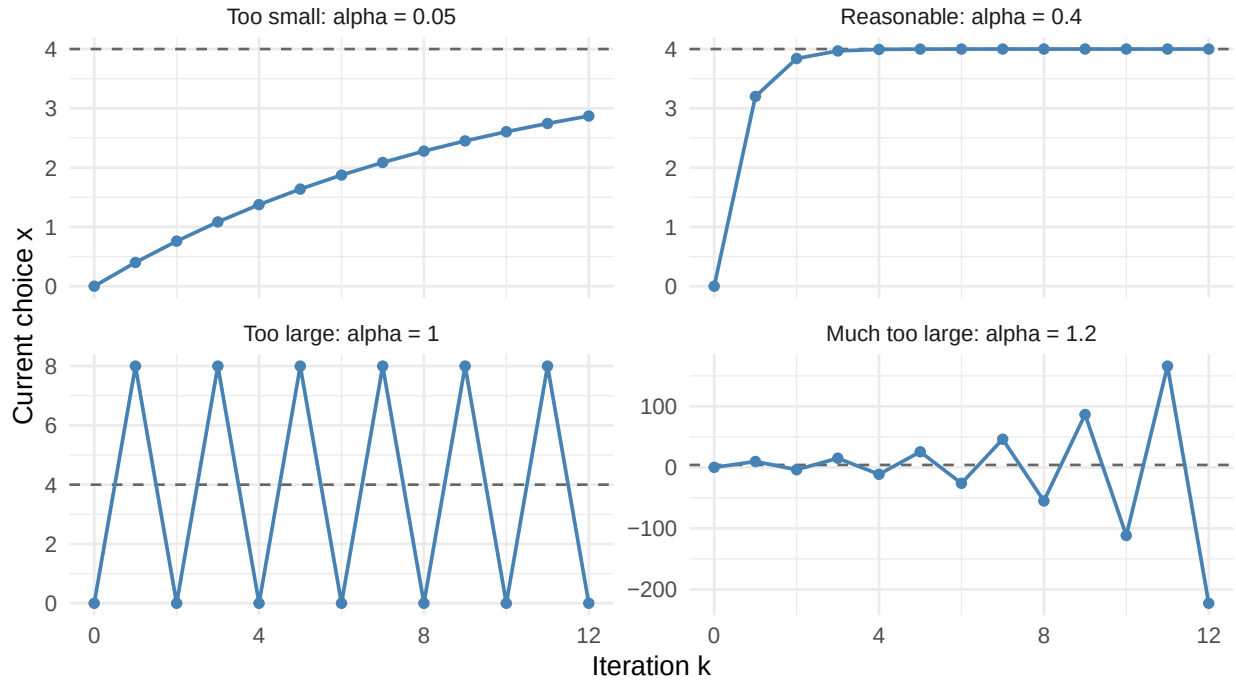


Figure 1: Gradient ascent on the same quadratic objective, starting at $x = 0$. The dashed line marks the maximizer $x = 4$. Note the different vertical scale in the divergence panel.

Line search: rather than choose α_k arbitrarily, an algorithm can search along the proposed direction for an acceptable step. For example, try a step and shrink it until it gives sufficient improvement: a decrease for minimization or an increase for maximization. This adapts the step size without requiring the second derivative or solving the line search exactly.

2.6 Summary: Comparing Rootfinding Algorithms

Method	Speed	Derivative Needed	Convergence Guarantee	Robustness	Common Use Cases
Bisection	Slow (linear)	No	Yes (if sign change)	Very robust	Bracketing roots in univariate problems
Newton's Method	Fast (quadratic)	Yes ($f'(x)$)	Local (if well-started)	Less robust	Solving FOCs, likelihood equations
Quasi-Newton Methods	Fast (superlinear)	No	Local (if well-started)	More robust than Newton	Large-scale optimization, machine learning

Choosing the Right Method

- Use **bisection** when you need guaranteed convergence and can bracket the root.
- Use **Newton's method** for fast convergence when you have a good initial guess and can compute derivatives.
- In practice, hybrid methods that combine these approaches are often employed to balance speed and robustness.

2.7 Multivariate optimization via rootfinding

2.7.1 Same idea, more decision variables

Move from $f(x)$ to $f(\mathbf{x})$, where $f : \mathbb{R}^n \rightarrow \mathbb{R}$. We still optimize **one objective**, such as profit or cost, but now choose n decision variables.

In one dimension, we optimize f by finding a root of $g(x) = f'(x)$. In multiple dimensions, the derivative becomes the **gradient**:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_n \end{bmatrix}.$$

Define $\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x})$. The first-order conditions for an interior optimum are $\mathbf{g}(\mathbf{x}^*) = \mathbf{0}$: every marginal change in the objective must be zero.

	One decision variable	Multiple decision variables
Objective	$f : \mathbb{R} \rightarrow \mathbb{R}$	$f : \mathbb{R}^n \rightarrow \mathbb{R}$
Function whose root we seek	$g(x) = f'(x)$	$\mathbf{g}(\mathbf{x}) = \nabla f(\mathbf{x})$
First-order condition	$g(x^*) = 0$	$\mathbf{g}(\mathbf{x}^*) = \mathbf{0}$

The objective is scalar-valued, but its gradient has n components. Finding a root of the gradient therefore means solving n **equations in n unknowns**. We are not trying to make the objective itself equal to zero.

2.7.2 Using the gradient to approach a stationary point

The gradient points in the direction of steepest increase.

For a fixed, small Euclidean step length, moving along the gradient gives the largest first-order increase in f . Moving against it gives the largest decrease. The one-variable gradient updates extend directly:

$$\begin{aligned} \text{Minimization: } \mathbf{x}_{k+1} &= \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k), \\ \text{Maximization: } \mathbf{x}_{k+1} &= \mathbf{x}_k + \alpha_k \nabla f(\mathbf{x}_k). \end{aligned}$$

The step size still matters: a direction that improves the objective locally can overshoot if we move too far.

Example: minimize the quadratic cost $f(x_1, x_2) = \frac{1}{2}(x_1^2 + 10x_2^2)$.

- Its gradient is $\mathbf{g}(\mathbf{x}) = [x_1, 10x_2]^\top$, so its unique root is $(0, 0)$, the cost minimizer.
- Cost rises more sharply in the x_2 direction, producing elliptical contours. Starting at $(4, 2)$ with $\alpha = 0.18$ gives $x_{1,k+1} = 0.82x_{1,k}$ and $x_{2,k+1} = -0.8x_{2,k}$: the second coordinate changes sign each step, while both approach zero.

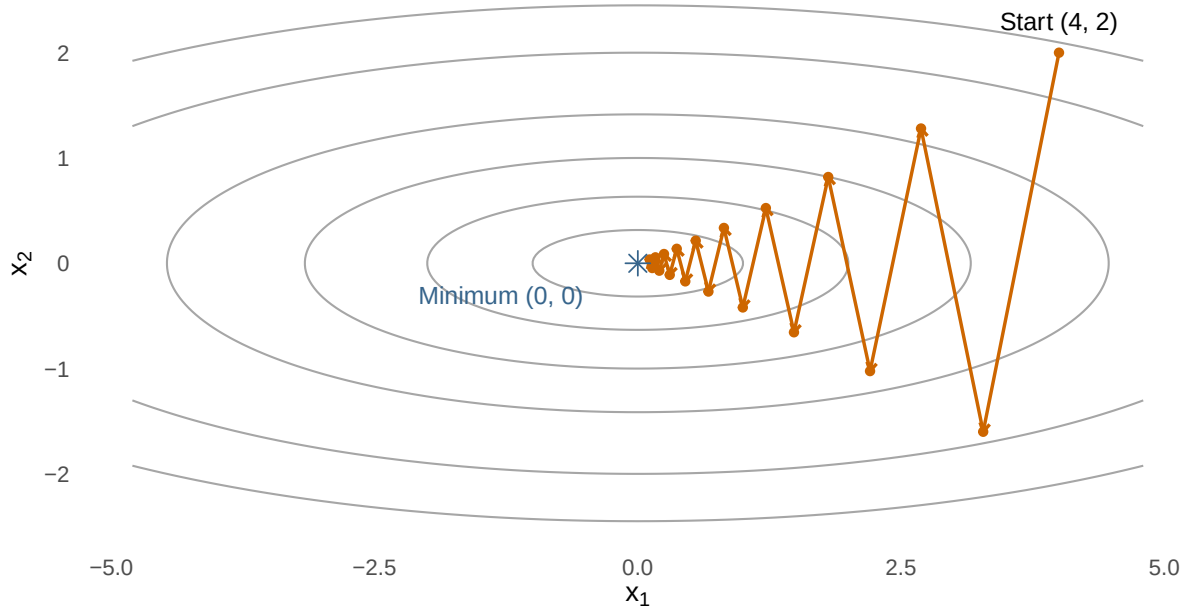


Figure 2: Gradient descent on elliptical cost contours. Each orange arrow is a step against the gradient. The iterates zig-zag toward the minimum at $(0, 0)$; a small gradient is the rootfinding target.

The same iterates on the cost surface show how the zig-zag moves down the bowl:

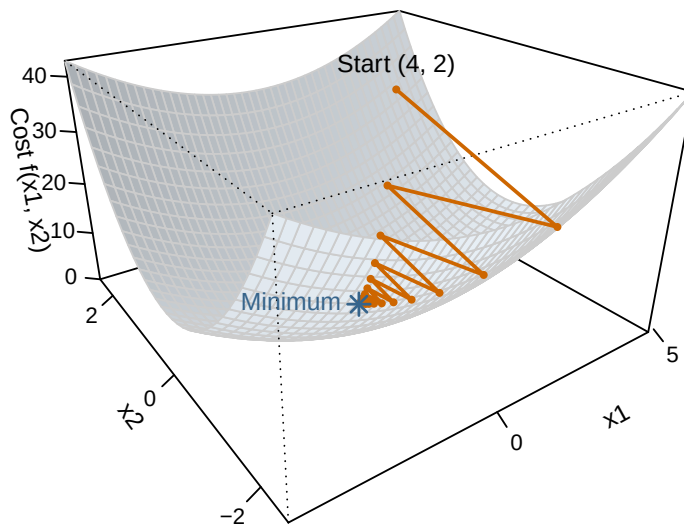


Figure 3: The same quadratic cost and gradient-descent iterates in three dimensions. Height is the objective value. Orange points are the iterates from the contour plot; segments connect successive iterates. The blue star marks the minimum.

2.7.3 Newton: find a root of the gradient

In one dimension, Newton uses g' to find a root of $g = f'$. In multiple dimensions, the derivative of $\mathbf{g} = \nabla f$ is the **Hessian**, $H(\mathbf{x}) = \nabla^2 f(\mathbf{x})$, whose entries are $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. It describes how the marginal returns change as we change the decisions.

The same local linear approximation gives

$$\mathbf{g}(\mathbf{x}_k + \mathbf{d}) \approx \mathbf{g}(\mathbf{x}_k) + H(\mathbf{x}_k)\mathbf{d}.$$

Set this approximation equal to zero to obtain Newton's update (when the Hessian is nonsingular):

$$\underbrace{x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}}_{\text{One dimension}} \quad \longrightarrow \quad \underbrace{\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k)}_{\text{Multiple dimensions}}.$$

- **Gradient:** which direction increases the objective most rapidly?
- **Hessian:** how steeply do marginal returns change in different directions?
- **Newton:** use both to choose a step toward zero marginal returns.

For the quadratic cost above, $H = \text{diag}(1, 10)$. Newton accounts for the different curvature in each direction and reaches $(0, 0)$ in one step, just as it did for our one-variable quadratic.

Both discussions target a **stationary point**. To conclude that it optimizes f , check the second-order conditions: a positive definite Hessian gives a strict local minimum, a negative definite Hessian a strict local maximum, and an indefinite Hessian a saddle point. A semidefinite Hessian is inconclusive. Away from the solution, Newton may still need a step-size or direction adjustment.

2.8 What actually goes wrong? (6 minutes)

Optimization software can return **an answer without returning the best answer**. Four issues matter in applied work:

- **Local versus global optima:** convergence to a local optimum does not establish global optimality. A small gradient and the right local curvature do not rule out a better solution elsewhere.
- **Starting values:** different $\mathbf{x}_0$ can lead to different solutions. Try several economically plausible starting values and compare objective values; agreement is useful evidence, not a proof of global optimality.
- **Scaling:** variables such as $x_1 \approx 0.001$ and $x_2 \approx 1,000,000$ can make step sizes, derivative calculations, and tolerances poorly balanced. Rescaling variables to comparable magnitudes can help without changing the underlying economic problem.
- **Stopping criteria:** “converged” usually means a numerical tolerance was satisfied, not that mathematics certified the solution. A tiny change in the parameters or objective can reflect stagnation; check the gradient or FOC residual as well.

Example: maximize $f(x) = -(x^2 - 1)^2 + 0.3x$. Run the same gradient-ascent rule, $x_{k+1} = x_k + 0.05f'(x_k)$, from $x_0 = -1.5$ and $x_0 = 1.5$. Both runs approach roots of $f'(x) = -4x(x^2 - 1) + 0.3$ with negative second derivatives, but the right-hand peak has a higher objective value.

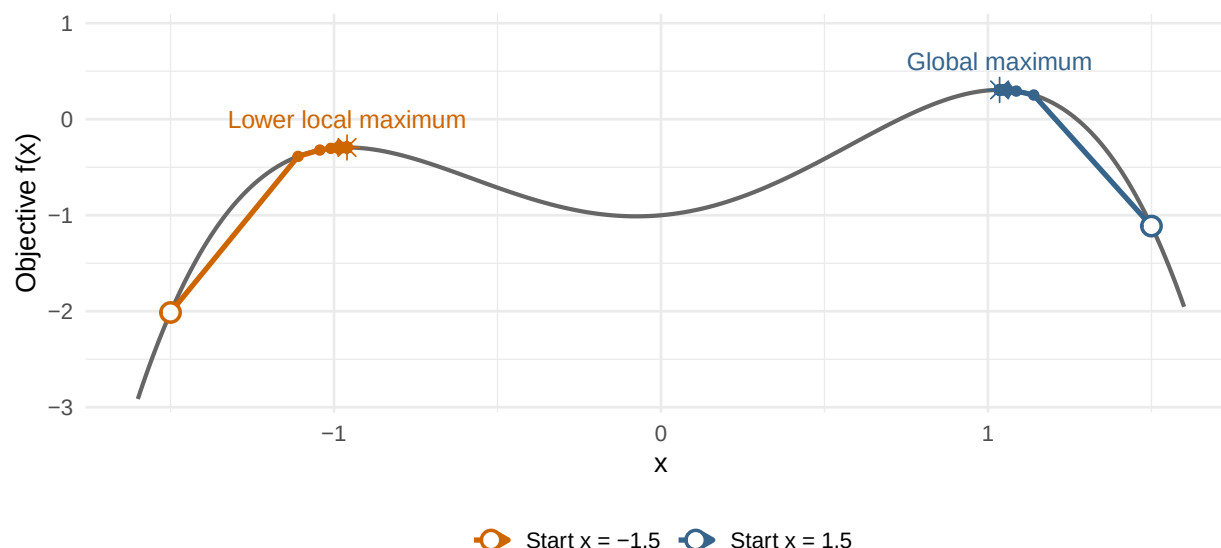


Figure 4: Same objective, algorithm, and step size; different starting values. Both runs converge to local maxima, but only the right-hand peak is the global maximum.

Before interpreting the economics, check the termination message, FOC residuals, local curvature, and sensitivity to starting values and scaling. Compare objective values across candidate solutions.

This section didn't work well for the class - too much for MS students and redundant for PhD students that have taken 735

3 Maximum Likelihood Estimation

Many econometric problems can be expressed as optimization problems. In particular, **Maximum Likelihood Estimation (MLE)** is a method of estimating unknown parameters by maximizing the likelihood of observing the sample data.

3.1 Likelihood function

Suppose we observe a sample of independent realizations $\{y_1, y_2, \dots, y_T\}$, each drawn from a distribution with density $f(y|\theta)$ parameterized by $\theta \in \Theta$.

For a concrete example, let's consider the Poisson.

The **likelihood function** is

$$L(\theta) = \prod_{t=1}^T f(y_t|\theta).$$

Because products of densities can be numerically unstable, we usually maximize the **log-likelihood function**:

$$\ell(\theta) = \log L(\theta) = \sum_{t=1}^T \log f(y_t|\theta).$$

3.2 Optimization problem

The MLE is defined as

$$\hat{\theta}_{MLE} = \arg \max_{\theta \in \Theta} \ell(\theta).$$

This is an **unconstrained optimization problem** whenever the parameter space $\Theta = \mathbb{R}^k$. In practice, standard optimization methods (Newton-Raphson, quasi-Newton, gradient ascent) are employed to find $\hat{\theta}_{MLE}$.

3.3 First- and second-order conditions

- **FOC (Score equation):**

$$\nabla_{\theta} \ell(\hat{\theta}) = 0.$$

- The term dates back to R.A. Fisher (1920s). He imagined the score as a running tally or “score card” of how much the data favors one parameter value over another.
- If the score is zero, the data have no incentive to move the estimate — you’ve “scored even.”

- **SOC (Information matrix):**

For $\hat{\theta}$ to be a maximum, the Hessian of $\ell(\theta)$ must be negative definite:

$$H(\hat{\theta}) = \nabla_{\theta}^2 \ell(\hat{\theta}) \prec 0.$$

- $\prec$ denotes negative definiteness.

- The sharper the log-likelihood curve, the more information the sample contains about the true parameter.
- A steep peak $\rightarrow$ small variance of $\hat{\theta}$.
- A flat peak $\rightarrow$ large variance.
- $\mathcal{J}(\theta)$ thus quantifies the precision of the estimator.

In large samples, the covariance matrix of $\hat{\theta}$ is approximated by the inverse of the **Fisher information matrix**:

$$\text{Var}(\hat{\theta}) \approx \mathcal{J}(\hat{\theta})^{-1}, \quad \mathcal{J}(\theta) = -\mathbb{E}[\nabla_{\theta}^2 \ell(\theta)].$$

3.4 Example: Normal distribution

Let $y_t \sim \mathcal{N}(\mu, \sigma^2)$ i.i.d. The log-likelihood is

$$\ell(\mu, \sigma^2) = -\frac{T}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=1}^T (y_t - \mu)^2.$$

- Maximizing with respect to μ yields

$$\hat{\mu} = \frac{1}{T} \sum_{t=1}^T y_t.$$

- Maximizing with respect to σ^2 yields

$$\hat{\sigma}^2 = \frac{1}{T} \sum_{t=1}^T (y_t - \hat{\mu})^2.$$

Both estimators emerge naturally from solving an optimization problem.

3.5 Economic applications

MLE is widely used in economics:

- Estimation of discrete choice models (logit, probit).
- Structural estimation of dynamic models
- Calibration of production or policy models where likelihood-based estimation can incorporate uncertainty.

MLE links optimization methods with statistical inference: the **same algorithms** (Newton, quasi-Newton, gradient descent) used to find optima in deterministic models also underpin parameter estimation in econometrics.

3.6 Numerically Solving MLE

In most economic applications, the log-likelihood function $\ell(\theta)$ does not admit a closed-form solution. We therefore rely on iterative optimization algorithms to find $\hat{\theta}_{MLE}$. These algorithms search over parameter values, guided by the slope (gradient) and sometimes curvature (Hessian) of the log-likelihood surface.

Gradient-based solvers:

- Newton-Raphson: Uses both the gradient and the exact Hessian. Converges quickly near the optimum but requires computing and inverting the Hessian at each step, which can be costly or unstable if the Hessian is poorly conditioned.
- Quasi-Newton methods (e.g. BFGS, BHHH): Approximate the Hessian rather than computing it directly. More stable and widely used in practice.

3.6.1 Newton-Raphson

1. Initialization: Start from an initial guess $\theta^{(0)}$ of length m .
2. Iterative update:
 - Compute the $m \times 1$ gradient $g^{(k)} = \nabla_{\theta} \ell(\theta^{(k)})$ and $m \times m$ Hessian $H^{(k)} = \nabla_{\theta}^2 \ell(\theta^{(k)})$.
 - Update parameters:

$$\theta^{(k+1)} = \theta^{(k)} - [H^{(k)}]^{-1} g^{(k)}.$$

3.6.2 Berndt-Hall-Hall-Hausman (BHHH)

The BHHH algorithm is a quasi-Newton method that approximates the Hessian using the outer product of gradients. It is particularly useful for MLE because it leverages the structure of the likelihood function.

1. Initialization: Start from an initial guess $\theta^{(0)}$.
2. Iterative update:
 - Compute the gradient $g^{(k)} = \nabla_{\theta} \ell(\theta^{(k)})$ which is an $m \times 1$.
 - Compute the outer product of gradients for each observation t :

$$B^{(k)} = \sum_{t=1}^T g_t^{(k)} g_t^{(k)'},$$

where $g_t^{(k)}$ is the gradient contribution from observation t . The dimensions of $B^{(k)}$ are $m \times m$.

- Update parameters:

$$\theta^{(k+1)} = \theta^{(k)} + [B^{(k)}]^{-1} g^{(k)}.$$

3.6.3 Broyden-Fletcher-Goldfarb-Shanno (BFGS)

The BFGS algorithm is a quasi-Newton method. It updates an approximation of the inverse Hessian at each iteration using only gradient information.

1. Initialization:
 - Start from an initial guess $\theta^{(0)}$.
 - Initialize an approximate inverse Hessian $H^{(0)}$ (often the identity matrix).
2. Iterative update:
 - Compute the gradient $g^{(k)} = \nabla_{\theta} \ell(\theta^{(k)})$.
 - Choose a search direction:

$$d^{(k)} = -H^{(k)} g^{(k)}.$$
 - Perform a line search to select a step size $\alpha^{(k)}$ that increases $\ell(\theta)$ sufficiently.
 - Update parameters:

$$\theta^{(k+1)} = \theta^{(k)} + \alpha^{(k)} d^{(k)}.$$
3. Update Hessian approximation:
 - Define $s^{(k)} = \theta^{(k+1)} - \theta^{(k)}$ and $y^{(k)} = g^{(k+1)} - g^{(k)}$.
 - Update inverse Hessian:

$$H^{(k+1)} = \left(I - \frac{s^{(k)} y^{(k)'}}{y^{(k)'} s^{(k)}} \right) H^{(k)} \left(I - \frac{y^{(k)} s^{(k)'}}{y^{(k)'} s^{(k)}} \right) + \frac{s^{(k)} s^{(k)'}}{y^{(k)'} s^{(k)}}.$$

Intuition

- Think of BFGS as learning the curvature of the log-likelihood surface on the fly.
- Newton's method jumps directly using the true curvature (the Hessian), but if that curvature estimate is noisy or expensive, the algorithm can diverge.
- BFGS instead builds a smoothed memory of past gradient steps to approximate curvature:

- $s^{(k)}$ tells the algorithm how parameters moved.
- $y^{(k)}$ tells the algorithm how gradients changed.
- Together, these updates gradually refine the Hessian approximation.
- As iterations proceed, the approximation converges to the true Hessian, so convergence accelerates.

Why Economists Like BFGS

- Robustness: Works well even when the likelihood is not globally concave.
- Efficiency: Avoids direct inversion of the Hessian, which is costly in high dimensions.
- Generality: Most econometric software (e.g., R's `optim()`, Stata's `ml`, MATLAB's `fminunc`) use BFGS as a default solver.

3.7 MLE Example

We'll estimate a simple logit model

$$\Pr(y_i = 1 \mid x_i) = p_i = \frac{1}{1 + \exp(-x_i' \beta)}.$$

Equivalently,

$$\text{logit}(p_i) = \log\left(\frac{p_i}{1 - p_i}\right) = x_i' \beta.$$

Likelihood

Given i.i.d. data $(y_i, x_i)_{i=1}^n$, the likelihood is

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Log-likelihood:

$$\ell(\beta) = \sum_{i=1}^n \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

Gradient (Score)

$$\nabla_{\beta} \ell(\beta) = \sum_{i=1}^n (y_i - p_i) x_i.$$

Hessian

$$\nabla_{\beta}^2 \ell(\beta) = - \sum_{i=1}^n p_i (1 - p_i) x_i x_i'.$$

4 Constrained Optimization

Let's start with the simple univariate case. The objective function has a single-dimension argument, x . Similarly, there is just one constraint and the function, $g(x)$ has a single dimension-argument, x .

4.1 Problem Setup

We consider the general problem:

$$\max_x f(x) \quad \text{s.t. } g(x) \leq b,$$

- $f(x)$: objective function
 - $g(x) \leq b$: inequality constraint
-

4.2 The Lagrangian and KKT Conditions

The **Lagrangian** is

$$\mathcal{L}(x, \lambda) = f(x) + \lambda(b - g(x))$$

where λ are Lagrange multipliers (also called **shadow values**). The shadow price tells you the marginal value (in terms of the objective) of relaxing the resource constraint by a unit.

The **KKT conditions** are:

1. **Stationarity**

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial f(x^*)}{\partial x} - \lambda \frac{\partial g(x^*)}{\partial x} = 0$$

2. **Primal feasibility**: the solution x^* must lie in the feasible set. You cannot propose an “optimal” solution that violates the budget, land, time, or other resource limits.

$$g(x^*) \leq b$$

3. **Dual feasibility**: the Lagrange multipliers (shadow prices) must be nonnegative. Why? Because if a constraint is binding, the multiplier represents the increase in the objective from relaxing that constraint.

$$\lambda \geq 0$$

4. **Complementary slackness**: Either the constraint binds and holds with equality and the shadow price is nonnegative **or** the constraint does not bind and the shadow price is 0.

$$\lambda(b - g(x^*)) = 0$$

4.3 Bordered Hessian (Second Order Condition)

The SOC's use the **bordered Hessian** to check the **curvature of f along feasible directions**. With constraints, we can't just look at the full Hessian. Some directions are **forbidden** (they would leave the feasible set). The bordered Hessian modifies the Hessian so that curvature is evaluated only along directions that *respect the constraints*. Imagine standing at the tangent plane of the feasible set at x^* . The bordered Hessian checks if f bends downward (for a max) or upward (for a min) *within that tangent plane*.

For one variable and one active constraint, the bordered Hessian is a 2×2 matrix:

$$H_B = \begin{bmatrix} 0 & -g_x(x^*) \\ -g_x(x^*) & \mathcal{L}_{xx}(x^*, \lambda^*) \end{bmatrix}.$$

(The minus sign comes from writing the constraint as $b - g(x) \geq 0$ in the Lagrangian.)

The determinant is

$$\det(H_B) = -[g'(x^*)]^2.$$

- For a **constrained maximum** with one binding constraint, we require $\det(H_B) < 0$.
- For a **constrained minimum**, $\det(H_B) > 0$.

Suppose we have a vector x with n dimensions and q constraints.

$$H_B = \begin{bmatrix} 0 & \nabla_x g(x^*)^\top \\ \nabla_x g(x^*) & \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) \end{bmatrix}.$$

- The **sign pattern of the principal minors** of H_B gives the curvature test:
 - For a maximum: determinants alternate in sign, starting with $(-1)^q$ starting from $q + 1$.
 - For a minimum: determinants are all $(-1)^{q+r}$ for the r th minor.

In constrained optimization: You must “filter out” the directions that are not feasible. The **bordered Hessian** does that filtering — it’s just a device to enforce that curvature is checked along the constraint surface.

In economics, global results often come from convexity assumptions:

- Concave objective + convex feasible set $\rightarrow$ every local maximum is global.
- This is why so much of microeconomic theory leans on convex analysis.

A set $C \subseteq \mathbb{R}^n$ is convex if, for any two points $x, y \in C$ and any $\theta \in [0, 1]$, the convex combination

$$z = \theta x + (1 - \theta)y$$

also belongs to C .

4.3.1 Example

Suppose we have a profit function $f(x_1, x_2)$ and a linear resource constraint:

$$\max_{x_1, x_2} f(x_1, x_2) = 10x_1 + 15x_2 - (x_1^2 + x_2^2) \quad \text{s.t. } x_1 + x_2 \leq 10$$

1. Find the optimum

The first-order conditions are

$$10 - 2x_1 - \lambda = 0,$$

$$15 - 2x_2 - \lambda = 0,$$

together with the binding constraint

$$x_1 + x_2 = 10.$$

From the first two equations:

$$2x_1 = 10 - \lambda, \quad 2x_2 = 15 - \lambda.$$

Solving for λ , setting equal, and subbing into the constraint,

$$10 - 2x_1 = 15 - 2x_2 \Rightarrow x_1 = x_2 - 5/2; \quad x_2 - 5/2 + x_2 = 10 \Rightarrow x_2 = 12.5/2 = 6.25$$

Substituting back:

$$x_1 = 6.25 - 2.5 = 3.75,$$

2. Check the SOC's

$\nabla g(x) = [11]$ so the borders are 1. Then, $\nabla f(x_1, x_2)$:

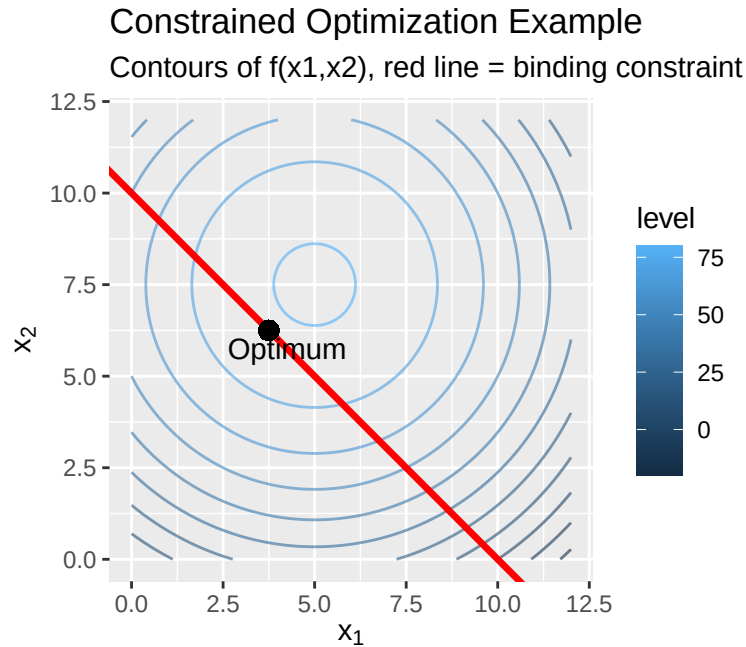
$$H_B = \begin{bmatrix} 0 & 1 & 1 \\ 1 & -2 & 0 \\ 1 & 0 & -2 \end{bmatrix}.$$

Starting from $q + 1 = 1 + 1 = 2$, must alternate sign starting with $(-1)^q = 1$:

$$|H_B| = \begin{vmatrix} 0 & 1 \\ 1 & -2 \end{vmatrix} < 0$$

and the full H_B

	[,1]	[,2]	[,3]
[1,]	0	1	1
[2,]	1	-2	0
[3,]	1	0	-2
[1]	-1		
[1]	4		



What is the shadow price of our constraint? $2(3.75) - 10 = 2.5 = \lambda$

What is the direction of steepest ascent?

- $f_{x_1}(x_1^*, x_2^*) = 10 - 2(3.75) = 2.5$
- $f_{x_2}(x_1^*, x_2^*) = 15 - 2(6.25) = 2.5$
- $\nabla g(x)$ points perpendicular to the constraint boundary (the “normal vector” to the feasible set).
- $\nabla f(x)$ points in the unconstrained steepest ascent direction.
- At the optimum on the boundary, these must line up — otherwise there would be a feasible direction in which f could still increase. One is just a rescaled version of the other, with the multiplier λ giving the scaling factor.

4.4 Envelope Theorem

- Once you set up the Lagrangian, the effect of parameter changes on the optimal value of the objective is given by multipliers.
- Example: In a consumer problem, $\frac{\partial V}{\partial I} = \lambda$ (marginal utility of income).
- Economists use this all the time in comparative statics, welfare analysis, and derivations of demand.

4.5 Duality

- The problem you solved is the primal: maximize the objective subject to constraints.
- Every primal problem has a dual problem, where the variables are the multipliers (the λ 's).
- The dual is not just a mathematical trick — it has deep economic meaning: it's a price system that supports the allocation chosen in the primal.

The Lagrangian itself is the bridge:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda(b - g(x)).$$

The dual function is

$$q(\lambda) = \max_x \mathcal{L}(x, \lambda).$$

The dual problem is

$$\min_{\lambda \geq 0} q(\lambda).$$

Economic interpretation: The dual chooses shadow prices λ so that the cost of resources (valued at λ) supports the primal allocation.

4.5.1 Example

The dual function is obtained by maximizing the Lagrangian over (x_1, x_2) for a given $\lambda \geq 0$:

$$q(\lambda) = \max_{x_1, x_2 \geq 0} \left[10x_1 + 15x_2 - x_1^2 - x_2^2 + \lambda(10 - x_1 - x_2) \right].$$

Stationarity conditions for the inner maximization:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x_1} = 10 - 2x_1 - \lambda = 0 & \Rightarrow x_1 = \frac{10 - \lambda}{2}, \\ \frac{\partial \mathcal{L}}{\partial x_2} = 15 - 2x_2 - \lambda = 0 & \Rightarrow x_2 = \frac{15 - \lambda}{2}. \end{aligned}$$

Plugging these $x_1(\lambda), x_2(\lambda)$ back into $\mathcal{L}$ gives the dual function. Compute each piece:

$$x_1 = \frac{10 - \lambda}{2}, x_2 = \frac{15 - \lambda}{2}.$$

$$q(\lambda) = 10 \frac{10 - \lambda}{2} - \left(\frac{10 - \lambda}{2} \right)^2 + 15 \frac{15 - \lambda}{2} - \left(\frac{15 - \lambda}{2} \right)^2 + \lambda(\lambda - 2.5).$$

The dual problem

$$\min_{\lambda \geq 0} q(\lambda).$$

Takeaways

- The dual problem says: find the resource price so that when agents maximize net benefit subject to that price, total demand for the resource just equals 10.
- The primal says: given 10 units of the resource, allocate across x_1 and x_2 to maximize net benefit.

Both yield the same value function, which is exactly the welfare-equilibrium duality economists care about.

5 Numeric Methods for Constrained Optimization

We will introduce constrained optimization numerical methods with the portfolio problem. In resource economics we often face **allocation problems under risk**. A manager (or policymaker) must decide how to allocate a fixed resource across several uses, where each use has uncertain payoffs.

Examples:

- An agricultural producer allocates land across crops with uncertain yields.
- A fishery regulator allocates harvest quotas across regions with uncertain stock growth.
- An energy portfolio manager invests in renewable projects with uncertain output.

These problems share a common structure:

- Decision variables $x = (x_1, \dots, x_n)$ represent fractions of the resource allocated to each activity.
- Each activity i has an **expected return** μ_i and **risk** Ω (variance and covariance with other activities).

5.1 Portfolio Problem

The manager wants to maximize expected return while managing risk, subject to a budget constraint $\sum_i x_i = 1$ and possibly non-negativity constraints $x_i \geq 0$ (no short selling).

We want to choose portfolio weights $x = (x_1, \dots, x_n)$ to maximize mean-variance utility:

$$\max_x U(x) = \mu^\top x - \frac{\lambda}{2} x^\top \Omega x$$

subject to:

- Budget constraint: $\sum_{i=1}^n x_i = 1$
- Non-negativity (no short selling, or negative land/harvest): $x_i \geq 0$

where:

- μ = vector of expected returns
- Ω = covariance matrix of returns
- λ = risk aversion parameter

This utility function is called **mean-variance utility** aka Markowitz utility. It captures the tradeoff between expected return and risk (variance of returns). The parameter λ controls how much the manager dislikes risk.

5.1.1 Solving the problem

Define the Lagrangian:

$$\mathcal{L}(x, \gamma, \nu) = \mu^\top x - \frac{\lambda}{2} x^\top \Omega x + \gamma (1 - \mathbf{1}^\top x) + \nu^\top x$$

where:

- γ is the multiplier for the budget constraint,
- $\nu \geq 0$ are multipliers for non-negativity constraints.

Intuition

- The problem is a **risk-return tradeoff**:
 - Higher return activities often come with higher variance.
 - Diversification reduces overall risk because covariance matters.
- In resource economics, this framework generalizes:
 - A farmer diversifying across crops is solving a **portfolio of land use** problem.
 - An energy planner investing in solar, wind, and hydro is solving a **portfolio of renewable energy projects**.
 - A conservation agency allocating budgets across programs faces the same tradeoff.
- The **shadow price** of the resource constraint tells us how much expected utility increases if we could expand the available resource slightly. In this case, it tells us about our marginal utility of wealth.

5.1.2 KKT Conditions

The KKT conditions are:

- Stationarity:

$$\nabla_x \mathcal{L} = \mu - \lambda \Omega x - \gamma \mathbf{1} + \nu = 0$$

- Primal feasibility:

$$\mathbf{1}^\top x = 1, \quad x \geq 0.$$

- Dual feasibility:

$$\nu \geq 0.$$

- Complementary slackness:

$$\nu_i x_i = 0 \quad \forall i.$$

Analytical Solution Without Inequality Constraints

Ignoring $x \geq 0$ (set $\nu = 0$), stationarity gives

$$\lambda \Omega x = \mu - \gamma \mathbf{1} \Rightarrow x(\gamma) = \frac{1}{\lambda} \Omega^{-1} (\mu - \gamma \mathbf{1})$$

Interpretation:

- First, note that the optimal allocation balances excess returns with the risk (weighted by risk aversion)
- allocate more to assets with higher excess returns $\mu_i - \gamma$ and lower variance (diagonal of Ω) and covariance (off-diagonal of Ω).
- Excess return is return above the shadow price γ of wealth. In this problem, the shadow price γ varies inversely with λ .
- If you are more risk averse (higher λ), the shadow price of wealth is lower because you value safety more. In other words, you need a higher expected return to compensate for risk.

If we impose the constraint, $\mathbf{1}^\top x = 1$, we can solve for γ (shadow price of wealth):

$$\mathbf{1}^\top \Omega^{-1} (\mu - \gamma \mathbf{1}) = \lambda \Rightarrow \gamma = \frac{\mathbf{1}^\top \Omega^{-1} \mu - \lambda}{\mathbf{1}^\top \Omega^{-1} \mathbf{1}}.$$

5.2 Numerical Example in R

```
library(ggplot2)

# Parameters
mu1 <- 0.10
Omega <- matrix(c(0.04,0.02,
                  0.02,0.09), 2, 2)
lambda <- 2

# Function: returns weights and gamma
project_nn <- function(mu, Omega, lambda){
  OmegaInv <- solve(Omega); one <- rep(1, length(mu))
  gamma <- as.numeric((t(one)%*%OmegaInv%*%mu - lambda) /
                    (t(one)%*%OmegaInv%*%one))
  x <- as.numeric((1/lambda) * OmegaInv %*% (mu - gamma*one))

  # enforce nonnegativity
  if(all(x >= -1e-10)) {
    return(list(x = pmax(x,0)/sum(pmax(x,0)), gamma = gamma))
  }
  if(x[1] < 0) return(list(x = c(0,1), gamma = gamma))
  if(x[2] < 0) return(list(x = c(1,0), gamma = gamma))
  list(x = pmax(x,0)/sum(pmax(x,0)), gamma = gamma)
}

# Sweep mu2
```

```

mu2_vals <- seq(0.001, 0.3, by=0.005)
results <- lapply(mu2_vals, function(m2) project_nn(c(mu1,m2), Omega, lambda))

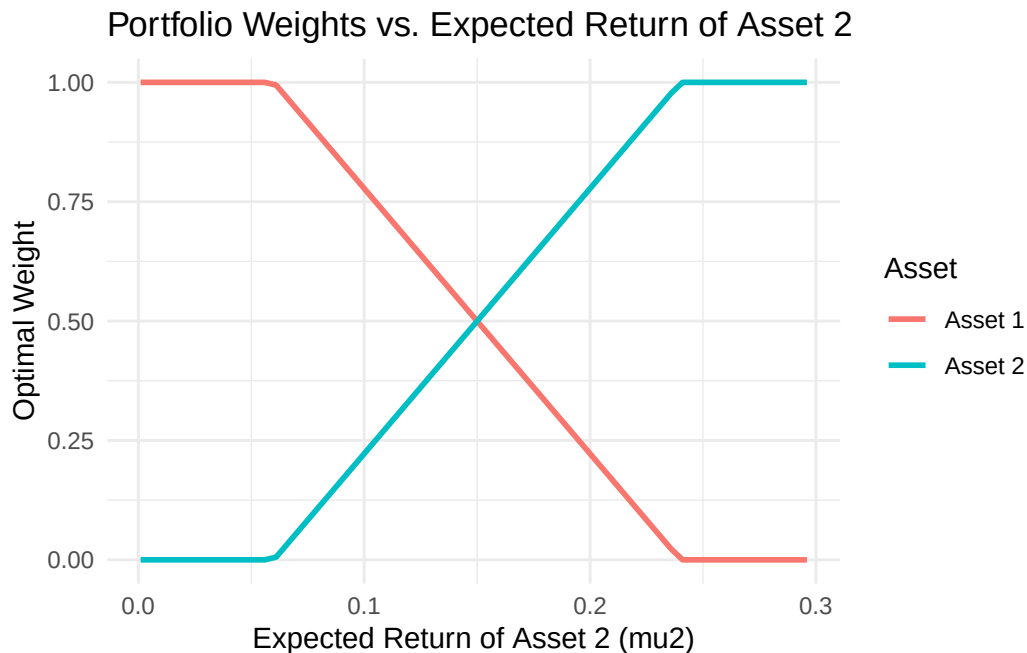
weights <- t(sapply(results, function(r) r$x))
gammas <- sapply(results, function(r) r$gamma)

df <- data.frame(mu2 = mu2_vals,
                 x1 = weights[,1],
                 x2 = weights[,2],
                 gamma = gammas)

# For teaching: take gamma at baseline mu2 = 0.15
gamma_ref <- df$gamma[10]

# Plot weights vs mu2 with vertical line at gamma
ggplot(df, aes(x=mu2)) +
  geom_line(aes(y=x1, color="Asset 1"), size=1) +
  geom_line(aes(y=x2, color="Asset 2"), size=1) +
  # geom_vline(xintercept = gamma_ref, linetype="dashed") +
  # annotate("text", x=gamma_ref, y=1.05,
  #         label=paste0("gamma = ", round(gamma_ref,3)),
  #         hjust=-0.1, vjust=0, angle=90) +
  labs(title="Portfolio Weights vs. Expected Return of Asset 2",
       x="Expected Return of Asset 2 (mu2)", y="Optimal Weight",
       color="Asset") +
  theme_minimal()

```

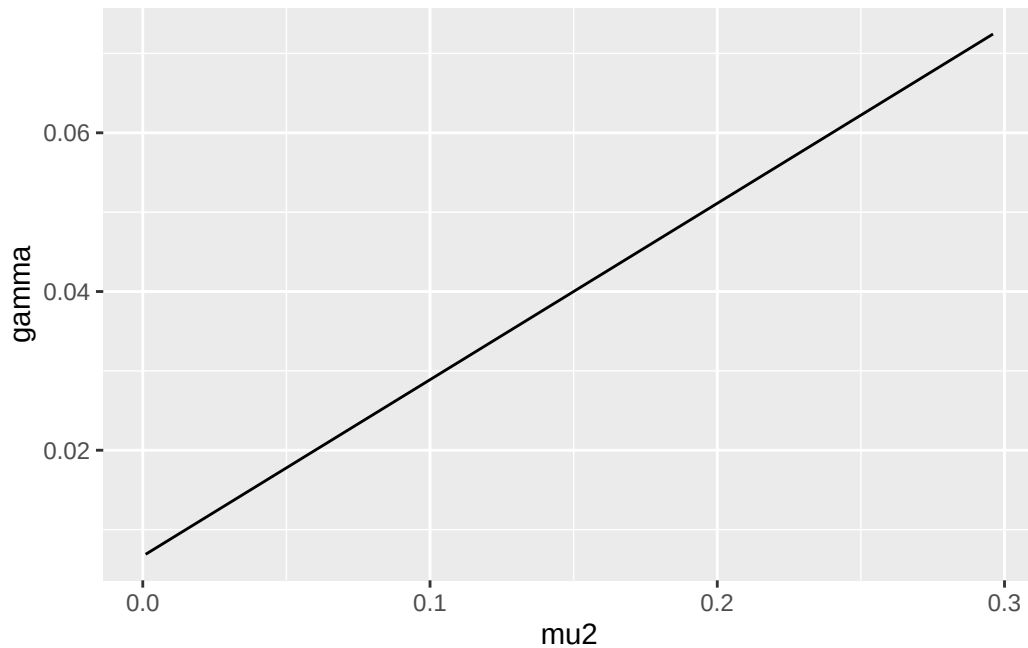


```

df |>
  ggplot(aes(x=mu2,y=gamma)) +

```

```
geom_line()
```



5.3 Sequential Least Squares Quadratic Programming

For larger allocation problems with many assets and constraints, we often need to use numerical solvers. One popular algorithm is **Sequential Least Squares Quadratic Programming (SLSQP)**.

The SLSQP idea

1. Start from a feasible x^k .
2. At each step, approximate the problem locally with a quadratic program (QP).
3. Solve the QP $\rightarrow$ gives a search direction d^k .
4. Update $x^{k+1} = x^k + \alpha d^k$ with a line search.
5. Repeat until gradients + constraints satisfy tolerances.

More details

1. Nonlinear problem setup
You have an objective $f(x)$ and constraints $g(x) = 0$ (budget) and $h(x) \geq 0$ (no short selling). Analytically, the KKT conditions require solving a system of equations involving gradients and multipliers.
2. Local quadratic approximation At the current guess x^k , SLSQP builds a quadratic model of the objective and linear models of the constraints:
 - Objective approximated by a quadratic (second-order Taylor expansion).

$$f(x) \approx f(x^k) + \nabla f(x^k)^\top (x - x^k) + \frac{1}{2} (x - x^k)^\top B^k (x - x^k)$$

- where B^k is an approximation of the Hessian (e.g. BFGS update).
- Constraints approximated by linearizing their gradients.

$$g(x) \approx g(x^k) + \nabla g(x^k)^\top (x - x^k)$$

$$h(x) \approx h(x^k) + \nabla h(x^k)^\top (x - x^k)$$

- This reduces the nonlinear program to a Quadratic Program (QP) at each step.

3. Solve the QP subproblem

The QP is much easier: quadratic objective + linear constraints.

Then find d to solve the QP subproblem:

$$\min_d; \nabla f(x^k)^\top d + \frac{1}{2} d^\top B^k d$$

subject to linearized constraints:

$$g(x^k) + \nabla g(x^k)^\top d = 0, \quad h(x^k) + \nabla h(x^k)^\top d \geq 0.$$

- The QP solution gives a step direction d^k .
4. Update with line search
To ensure stability, SLSQP does a line search along d^k :

$$x^{k+1} = x^k + \alpha d^k, \quad 0 < \alpha \leq 1$$

- It chooses α to reduce the merit function (a mix of objective + penalties for constraint violation).
 - Then it updates B^k (the Hessian approximation).
5. Iterate until convergence
Repeat until the gradients are small and the constraints are satisfied (up to tolerance).
- Stationarity holds: $\nabla f(x^*) + \sum \lambda_j \nabla g_j(x^*) = 0$.
 - Constraints satisfied: $g(x^*) = 0, h(x^*) \geq 0$.
 - Complementary slackness: $\lambda_j h_j(x^*) = 0$.

5.3.1 Roles of gradient and Hessian

- Gradient term $\nabla f(x^k)$ gives the local direction of steepest descent/ascent.
- If we only used this term, the subproblem would be a linear program:

$$\min_d; \nabla f(x^k)^\top d$$

- subject to linearized constraints. That would always push to an extreme corner of the feasible region (like simplex in LP).
- Hessian term B^k curves the model of the objective.
- It tells the solver how the slope changes as you move, i.e. how “sharp” or “flat” the function is in different directions.
- This ensures the step d is not just feasible and descending, but also “shaped” to anticipate the curvature of f .

In optimization language:

- Gradient = first-order information (direction of decrease).
- Hessian = second-order information (how big of a step you can take in each direction without overshooting).

6 Simulated Method of Moments: A Static Abatement Model

We consider a simple static environmental economics problem. A firm emits pollution but can **abate** emissions at a cost. The regulator imposes a penalty on residual emissions, which induces the firm to choose a positive abatement level.

6.1 Model Setup

- Each firm i has baseline emissions e_i^0 before abatement.
- The firm chooses abatement $a_i \geq 0$.
- True abatement costs are quadratic in a_i :

$$C(a_i; \theta) = \theta_1 a_i + \frac{1}{2} \theta_2 a_i^2$$

where $\theta_1, \theta_2 > 0$.

- Actual residual emissions after abatement are stochastic:

$$e_i = e_i^0 - a_i + \varepsilon_i,$$

with $\varepsilon_i \sim N(0, \sigma^2)$, independent across firms.

- The regulator imposes a per-unit penalty P on realized residual emissions. The expected penalty to firm i is

$$P \cdot \mathbb{E}[\max\{0, e_i^0 - a_i + \varepsilon_i\}].$$

- Firm i solves

$$\min_{a_i \geq 0} C(a_i; \theta) + P \cdot \mathbb{E}[\max\{0, e_i^0 - a_i + \varepsilon_i\}].$$

Suppose we could move the expectation operator inside the max function (we cannot) and assume that $e_i^0 - a_i$ is always positive (we cannot). Solve the problem.

6.2 Deriving the FOC

6.2.1 Step 1. Firm's problem

The firm minimizes expected cost:

$$\min_{a \geq 0} \theta_1 a + \frac{1}{2} \theta_2 a^2 + P \cdot \mathbb{E}[\max\{0, e_0 - a + \varepsilon\}],$$

where $\varepsilon \sim N(0, \sigma^2)$.

6.2.2 Step 2. Derivative of the cost function

The marginal cost of abatement is:

$$\frac{\partial}{\partial a} \left(\theta_1 a + \frac{1}{2} \theta_2 a^2 \right) = \theta_1 + \theta_2 a.$$

6.2.3 Step 3. Derivative of the expected penalty

Inside the max function:

$$g(a, \varepsilon) = \max\{0, e_0 - a + \varepsilon\}.$$

The derivative with respect to a is:

$$\frac{\partial g}{\partial a} = \begin{cases} -1, & \text{if } e_0 - a + \varepsilon > 0, \\ 0, & \text{otherwise.} \end{cases}$$

So:

$$\frac{\partial}{\partial a} \mathbb{E}[g(a, \varepsilon)] = \mathbb{E}\left[\frac{\partial g}{\partial a}\right] = -\Pr(e_0 - a + \varepsilon > 0).$$

6.2.4 Step 4. Probability expression

Since $\varepsilon \sim N(0, \sigma^2)$:

$$\Pr(e_0 - a + \varepsilon > 0) = \Pr(\varepsilon > a - e_0) = 1 - \Phi\left(\frac{a - e_0}{\sigma}\right) = \Phi\left(\frac{e_0 - a}{\sigma}\right),$$

where $\Phi(\cdot)$ is the standard normal CDF.

6.2.5 Step 5. First-order condition

Setting marginal cost = expected marginal benefit:

$$\theta_1 + \theta_2 a = P \cdot \Phi\left(\frac{e_0 - a}{\sigma}\right).$$

Then plugging in our true theta values $\theta_1 = 2$, $\theta_2 = 0.5$, $P = 10$, $\sigma = 1$, and $e_0 = 10$:

$$1 + 0.5a = 10 \cdot \Phi(e_0 - a).$$

When $e_0 - a$ is large, $\Phi(\cdot) \approx 1$, so $a \approx 18$. When $e_0 - a$ is small or negative, $\Phi(\cdot) \approx 0$, so $a \approx -4$ (infeasible). The solution is somewhere in between.

6.3 Economic Intuition

- The firm trades off **abatement cost** (increasing and convex in a_i) against the **expected penalty** (decreasing in a_i).
- Optimal abatement a_i^* rises when:
 - Penalty P is higher,
 - Baseline emissions e_i^0 are larger,
 - The variance of ε_i is lower (less uncertainty in residuals).
- In practice, the expectation over ε_i has no simple closed form. Thus, we approximate it by Monte Carlo simulation.

6.4 Simulated Method of Moments (SMM)

6.4.1 Why simulation?

- Analytical expressions for expected penalties are unavailable.
- However, we can simulate many ε_i draws, approximate the expectation, and solve for optimal a_i numerically.
- This creates a “black box” mapping from parameters $\theta = (\theta_1, \theta_2, \sigma)$ to simulated data $\{a_i\}$.

6.5 Observed data and empirical moments

Suppose we observe a sample of N firms with (e_i^0, a_i) pairs. We can compute **empirical moments** such as:

- Mean abatement:

$$m_1 = \frac{1}{N} \sum_{i=1}^N a_i$$

- Variance of abatement:

$$m_2 = \frac{1}{N} \sum_{i=1}^N (a_i - \bar{a})^2$$

- Covariance of abatement with baseline emissions:

$$m_3 = \frac{1}{N} \sum_{i=1}^N (a_i - \bar{a})(e_i^0 - \bar{e}^0).$$

These provide identifying information about θ .

6.6 Simulated moments

Given a candidate parameter vector θ :

1. For each firm i , simulate shocks $\varepsilon_{i,s}$ using common random numbers.
2. Solve the firm’s optimization problem for optimal abatement $a_i(\theta)$.
3. Compute **simulated moments** $\psi(\theta) = (\psi_1(\theta), \psi_2(\theta), \psi_3(\theta))$ analogous to m_1, m_2, m_3 .

6.7 The SMM criterion

The estimator chooses θ to minimize the distance between data and simulated moments:

$$\hat{\theta} = \arg \min_{\theta \in \Theta} [m - \psi(\theta)]^\top W [m - \psi(\theta)]$$

- m = vector of empirical moments.
- $\psi(\theta)$ = vector of simulated moments.
- W = weighting matrix. We start with the identity matrix, then can improve efficiency using the inverse of the estimated covariance of the moments.

This is a **constrained optimization problem**: $\theta_1, \theta_2, \sigma > 0$. The SMM criterion is generally nonlinear and must be solved numerically.

6.8 Economic Interpretation

- θ_1 : linear cost of abatement, shifts the level of optimal abatement.
- θ_2 : curvature of cost; higher θ_2 implies rapidly increasing marginal costs, reducing optimal abatement variance.
- σ : noise in residual emissions. Higher σ makes penalties more uncertain, dampening abatement incentives.

By matching simulated and empirical moments, we infer the structural cost parameters from observed behavior.

6.9 Implementation Highlights

- Use **common random numbers (CRNs)**: fix random shocks across evaluations so the SMM objective is smooth.
 - Inner loop: for each firm, solve a one-dimensional minimization in a_i using `optimize()` in R.
 - Outer loop: minimize the SMM objective using a constrained solver such as `nloptr`.
 - Diagnostic checks:
 - Compare simulated moments at $\hat{\theta}$ vs empirical moments.
 - Plot the objective surface in 2D slices around $\hat{\theta}$.
 - Sensitivity of $\hat{\theta}$ to number of simulation draws and weighting matrix.
-

6.10 Key Takeaways

- SMM allows us to estimate structural parameters in models where expectations are analytically intractable.
- The method is flexible: as long as we can simulate from the model, we can estimate it.
- The abatement example is intentionally simple, but the same framework scales to more complex resource and environmental economics problems:
 - groundwater extraction under stochastic recharge,
 - fishery harvest with stochastic stock dynamics,
 - permit allocation under uncertainty.